Experiment 1:

Install Hadoop and Implement the following file management tasks in Hadoop:
Adding files and directories
Retrieving files
Deleting files and directories.
Hint: A typical Hadoop workflow creates data files (such as log files) elsewhere and copies them into HDFS using one of the above command line utilities.

Installation of Hadoop Single Node Cluster:
1. Prerequisites:
• A Laptop or Desktop with 32 or 64 bit with window (Linux operating system when using VM)
• Minimum of 2 Gb ram
• Minimum of 100 gb Hard Disk
• Minimum VGA is required
2. JRE 1.8 — Offline installer for JRE
3. Java Development Kit — 1.8
4. A Software for Un-Zipping like 7Zip or Win Rar
5. Download Hadoop zip * I am using Hadoop-1.2.0
Steps of Installing of Hadoop Single Node Cluster Configuration.
1. Downloading the software Required
2. Untar the software
3. Bashrc configurations
4. Hadoop Configuration File
5. Share public key to localhost
6. Formatting the name node
7. Starting Hadoop Daemons
8. Checking the working Hadoop Daemons
Step 1: SOFTWARES REQUIRED
• Hadoop 1.2.0-bin.tar.gz • Jdk 7u67-linux-i586.tar.gz
Link for JDK http://www.oracle.com/technetwork/java/javase/downloads/ind ex.html
Link of Hadoop : http://mirror.fibergrid.in/apache/hadoop/common/hadoop-1.2.0/
Step 2: Untar the software
• Open the terminal window
• Type the command : cd Desktop
• Type ls command
• tar –zxvf jdk-7u67-linux-i586.tar.gz
• tar –zxvf hadoop-1.2.0-bin.tar.gz
Step 3: Bashrc configurations
Open the terminal type the command
sudo gedit ~/.bashrc

export JAVA_HOME=/home/user/Desktop/jdk1.7.0_67
export PATH=$PATH:$JAVA_HOME/bin
export HADOOP_HOME=/home/user/Desktop/hadoop-1.2.0
export PATH=$PATH:$HADOOP_HOME/bin
export PATH=$PATH:$HADOOP_HOME/sbin

Open new terminal and check for java version and Hadoop version.
Step 4: Hadoop Configuration Files
1. Core-site.xml
(Configuration setting for hadoop core, such as I/O setting that are common to HDFS and Mapreduce)
2.Hdfs-Site.xml
(Configuration setting for HDFS daemons the name node, Secondary name node and data node and Replication factor as well)
3. Mapred-site.xml

(configuration setting for MapReduce daemons: the job tracker and the task tracker)
4. Hadoop-Env_sh
(Environment variables that are used in the scripts to run Hadoop)
• Open the new terminal window
• Go to the hadoop folder
• cd Desktop • cd Hadoop (Press tab) for path
• cd conf
• Type ls
• All the above mentioned files are listed
• Go to terminal and type

sudo gedit core-site.xml

```
<configuration>
 <property>
 <name>fs.default.name</name>
 <value>hdfs://localhost:9000</value>
 </property>
 </configuration>
```

• Go to terminal and type

sudo gedit hdfs-site.xml

```
<configuration>
 <property>
 <name>dfs.replication</name>
 <value>1</value>
 <name>dfs.name.dir</name>
 <value>/home/user/Desktop/hadoop-1.2.0/name/data</value>
 </property>
 </configuration>
```

• Go to terminal and type

sudo gedit mapred-site.xml

```
<configuration>
 <property>
 <name>mapred.job.tracker</name>
 <value>hdfs://localhost:9001</value>
 </property>
 </configuration>
```

• Go to terminal and type

sudo gedit hadoop-env.sh

export JAVA_HOME=/home/user/Desktop/jdk1.7.0_67

Step 5: SSh Configuration

• sudo apt-get install ssh
- isa command that installs the SSH package on a Ubuntu VPS. The SSH package uses the open source. Open SSH suite to provide encryption for network services like remote login and remote file transfers.
• ssh-keygen -t rsa -P ""

Sharing the public key with host

• ssh-copy-id –i ~/.ssh/id_rsa.pub user@ubuntuvm and check
• ssh localhost (It should not ask password)

Step 6: Formatting the Name Node

• $ hadoop namenode –format
• format a new distributed file system
• Process creates an empty file system for creating the storage directories

Step 7 & 8: Starting Hadoop Daemons

Open the terminal window and type
$ start-all.sh
and type
$ jps

- *user@ubuntuvm:~$ start-all.sh*
- *Warning: $HADOOP_HOME is deprecated.*
- *starting namenode, logging to /home/user/Downloads/hadoop 1.2.0/libexec/../logs/hadoop-user-namenode-ubuntuvm.out*
- *localhost: datanode running as process 2978.*
- *localhost: secondarynamenode running as process 3123.*
- *jobtracker running as process 3204.*
- *localhost: tasktracker running as process 3342.*
- *user@ubuntuvm:~$ jps*
- *4020 Jps*
- *3342 TaskTracker*
- *3204 JobTracker*
- *3123 SecondaryNameNode*
- *3606 NameNode*
- *2978 DataNode*

Adding files and directories:

1. Adding Directories:
$ hdfs dfs –mkdir <path>
Creates a directory named path in HDFS. Creates any parent directories in path that are missing e. g. , mkdir pinLinux.

hdfs dfs -mlkdir /home
Similarly create many folders
hdfs dfs -mkdir /datasets
hdfs dfs -mkdir /titanicdata
hdfs dfs -mkdir /otherdataset

2. Adding files:
hdfs dfs –put <localSrc> <destination>

Copies the file or directory from the local file system identified by localSrc to dest within the DFS.
hdfs dfs -put file:///home/hadoop/sh.txt /
hdfs dfs –put file:///home/hadoop/work/datasets/Titanic.csv /titanicdata

//Now you have stored data in HDFS. Now verify this data stored in hdfs or not

hdfs dfs –ls

Lists the contents of the directory specified by path, showing the names, permissions, owner, size and modification date for each entry.
hdfs dfs -ls /
hdfs dfs -ls /Titanicdata

//If you want to display recursively inside that folder, use -lsr or -ls –R

3. Retrieving files:
hdfs dfs -cat <filename>
Displays the contents of filename on stdout.
hdfs dfs -cat /sh.txt

4. Deleting files and directories.
hdfs dfs –rm <path>
Removes the file or empty directory identified by path.

hdfs dfs -rm /data.txt
hdfs dfs –rmr <path>
Removes the file or directory identified by path. Recursively deletes any child entries i.e., files or sub directories of path.
hdfs dfs -rmr /home/

Experiment 2:
Develop a MapReduce program to implement Matrix Multiplication.

Program:

MatrixMultiply:

```
import org.apache.hadoop.conf.*;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;
public class MatrixMultiply {
public static void main(String[] args) throws Exception {
if (args.length != 2) {
System.err.println("Usage: MatrixMultiply <in_dir> <out_dir>");
System.exit(2);
}
Configuration conf = new Configuration();
// M is an m-by-n matrix; N is an n-by-p matrix.
conf.set("m", "1000");
conf.set("n", "100");
conf.set("p", "1000");

Job job = new Job(conf, "MatrixMultiply");
job.setJarByClass(MatrixMultiply.class);
job.setOutputKeyClass(Text.class);
job.setOutputValueClass(Text.class);
job.setMapperClass(Map.class);
job.setReducerClass(Reduce.class);
job.setInputFormatClass(TextInputFormat.class);
job.setOutputFormatClass(TextOutputFormat.class);
FileInputFormat.addInputPath(job, new Path(args[0]));
FileOutputFormat.setOutputPath(job, new Path(args[1]));
job.waitForCompletion(true);
}
}
```

Mapper:

```
import org.apache.hadoop.conf.*;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;
import java.io.IOException;
public class Map extends Mapper<LongWritable, Text, Text, Text> {
@Override
public void map(LongWritable key, Text value, Context context)
throws IOException, InterruptedException {
Configuration conf = context.getConfiguration();
```

```
int m = Integer.parseInt(conf.get("m"));
int p = Integer.parseInt(conf.get("p"));
String line = value.toString();

String[] indicesAndValue = line.split(",");
Text outputKey = new Text();
Text outputValue = new Text();
if (indicesAndValue[0].equals("M")) {
for (int k = 0; k < p; k++) {
outputKey.set(indicesAndValue[1] + "," + k);

outputValue.set(indicesAndValue[0] + "," + indicesAndValue[2]+ "," +
indicesAndValue[3]);

context.write(outputKey, outputValue);
}
} else {

for (int i = 0; i < m; i++) {
outputKey.set(i + "," + indicesAndValue[2]);
outputValue.set("N," + indicesAndValue[1] + ","
+ indicesAndValue[3]);
context.write(outputKey, outputValue);
}
}
}
}
```

Reducer:

```
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;
import java.io.IOException;
import java.util.HashMap;
public class Reduce extends Reducer<Text, Text, Text, Text> {
@Override
public void reduce(Text key, Iterable<Text> values, Context context)
throws IOException, InterruptedException {
String[] value;
//key=(i,k),
//Values = [(M/N,j,V/W),..]
HashMap<Integer, Float> hashA = new HashMap<Integer, Float>();
HashMap<Integer, Float> hashB = new HashMap<Integer, Float>();
for (Text val : values) {
value = val.toString().split(",");
if (value[0].equals("M")) {
hashA.put(Integer.parseInt(value[1]), Float.parseFloat(value[2]));
} else {
hashB.put(Integer.parseInt(value[1]), Float.parseFloat(value[2]));
}
}
int n = Integer.parseInt(context.getConfiguration().get("n"));
float result = 0.0f;
float m_ij;
```

```
float n_jk;
for (int j = 0; j < n; j++) {
m_ij = hashA.containsKey(j) ? hashA.get(j) : 0.0f;
n_jk = hashB.containsKey(j) ? hashB.get(j) : 0.0f;
result += m_ij * n_jk;
}
if (result != 0.0f) {
context.write(null,new Text(key.toString() + "," + Float.toString(result)));
}
}
}
```

Execution Procedure:

1.create java project and save with appropriate project name

2.Right click on project and create 3 java class (Mainfile, Mapper, Reducer)

3.Add Jar files-right click on project ,
a.choose build path option
b.choose configure build path
c.click on add external jars
d.path - Desktop/hadoop-java-jars - select all by pressing ctrl+A , click ok

4.Add implementation code in all three java files

---> Create jar file (right click proj name, export, jar creation)

5.commands for execution - open new terminal

6.hadoop dfs -mkdir /input - create directory to store all dataset files in HDFS-
hadoop dfs -ls /input -displays files inside input directory

7.hadoop dfs -put local file system path /input/ - copies file from local file system to HDFS

8.hadoop jar path-to-jarfile mainclass /path-to-inputfile /output

copy paste path wherever required

9.to check output there are 2 ways :

a.check output from browser --- open firefox browser type in url
localhost:50070- browse filesystem choose output directory and open part-00000 file

b.use cat command

hadoop dfs -ls /output --displays contents of output directory
hadoop dfs -cat /output/part-00000 - displays contents of this file

Hint: In main Class change

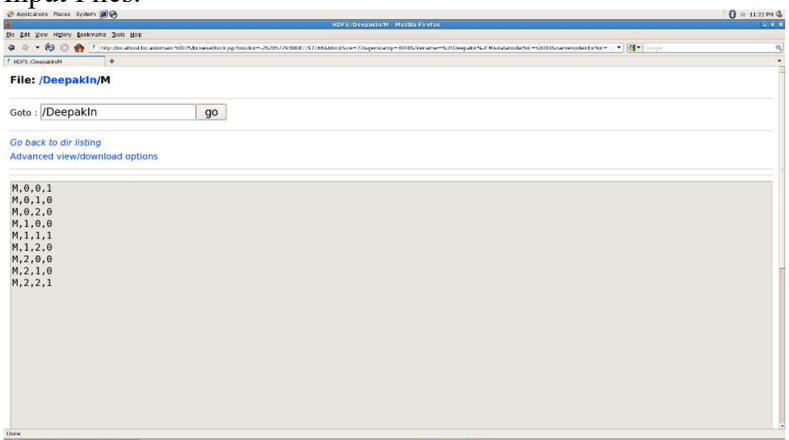
class name in JobConf() function
class name in setMapperClass() function
class name in setReducerClass() function
main method change in run() with constructor name

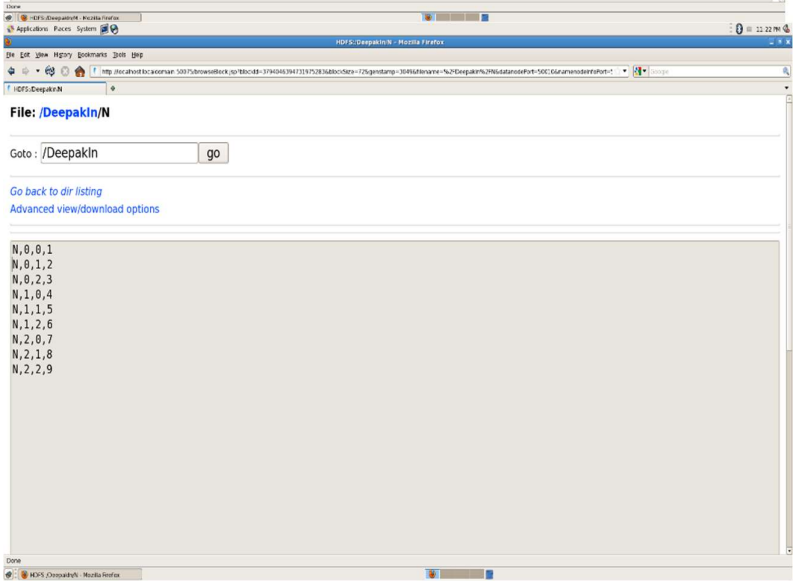
Data:

A=[1,0,0,0,1,0,0,0,1]
B=[1,2,3,4,5,6,7,8,9]

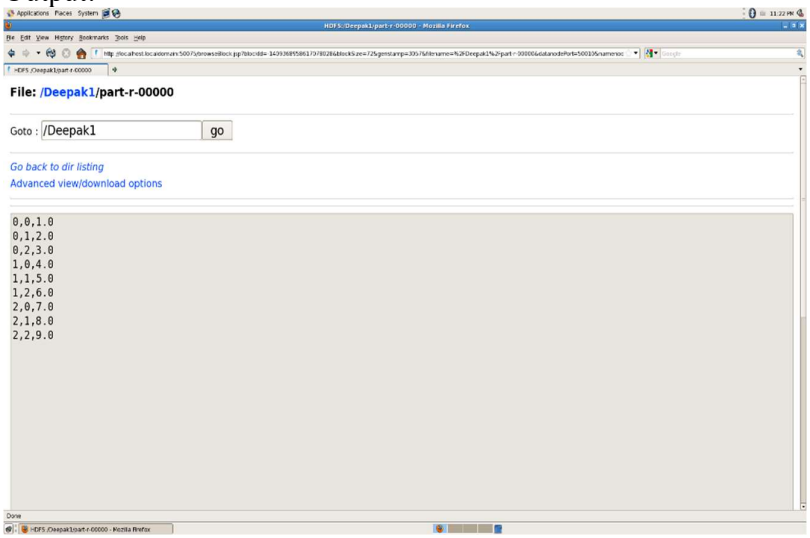
Result : [1,2,3,4,5,6,7,8,9]

Input Files:





Output:


File: /Deepak1/part-r-00000
Goto : /Deepak1
go
Go back to dir listing
Advanced view/download options
0,0,1.0
0,1,2.0
0,2,3.0
1,0,4.0
1,1,5.0
1,2,6.0
2,0,7.0
2,1,8.0
2,2,9.0